Lab 06 Template

Part 01

1. **Upload python code for babyFeistel cipher as `<netid>_part01.py`**
   (5 points)

2. **Final 4-bit ciphertext.**
   (5 points)

   Ans) 1101

3. **Screenshot of encrypting.**
   (5 points)

```
cpre3310@cpre3310:~/lab06/part01$ python3 part01_skel.py
4-bit input to encode: 0100
4-bit key: 1010
Round 0:  0001
Round 1:  0111
Round 2:  1101
```

Part 02

4. **Include a screenshot of the two keys and the matching ciphertexts and plaintexts in your lab report.**
   (5 points)

```
cpre3310@cpre3310:~/lab06/part02$ python3 part02_mitm_2DES_skel.py part02_ciphertext.txt
Starting MITM attack...
Matching intermediate value found!
Found Key 1 (stripped parity): 0000000000000200
Found Key 2 (stripped parity): 0000000002020202
Keys written to recoveredKeys.txt

The given ciphertext is:  c1b9303f9242d623
The ciphertext from doubleDesEncrypt is:  c1b9303f9242d623
The given plaintext is:  b'Computer'
The plaintext from doubleDesDecrypt is:  b'Computer'
```

5. **What is a known flaw with the block cipher mode being used?**
   (5 points)

   Ans) The block cipher mode that we have used over here is the electronic codebook and the major flaw of using that is the identical plaintext blocks produce identical ciphertext blocks when they are encrypted with the same key which reveals patterns inside the plaintext and makes the encryption vulnerable to analysis since the repeated data structure may still remain visible in the ciphertext, so as a result the ECB does not provide strong confidentiality for the structured or repetitive data.

6. **Why is known plaintext important in a meet-in-the-middle attack? What role does it play in recovering the encryption keys?**
   (10 points total, 5 points each question)

   Ans) It is important for this attack since it allows the attacker to compare the intermediate encryption values, so now the attacker encrypts known plaintext by using all the possible values of the first key and stores the intermediate result and after that the attacker can decrypt the known ciphertext by using all the possible values of the second key and checks for any matches with the stored intermediate values and when a match occurs, it suggests that the corresponding key-pair could be the correct keys that were basically used in the encryption process, which basically reduces the search space compared to brute force, making it possible to recover the keys faster than trying every combination directly.

7. **Could this attack be extended to 2-key 3DES? What about 3-key 3DES? What differences or additional challenges would arise in attacking 3DES with a similar approach?**
   (10 points total, 3.33 points each question)

   Yes, we can extend this attack to 2-key 3DES, but the attack will become more complex in this case and may require more computation and memory. Also, over here the 2-key 3DES basically uses 2 independent keys and 3 encryption operations, so the attacker must deal with additional intermediate states, which in turn increases the cost of the attack.

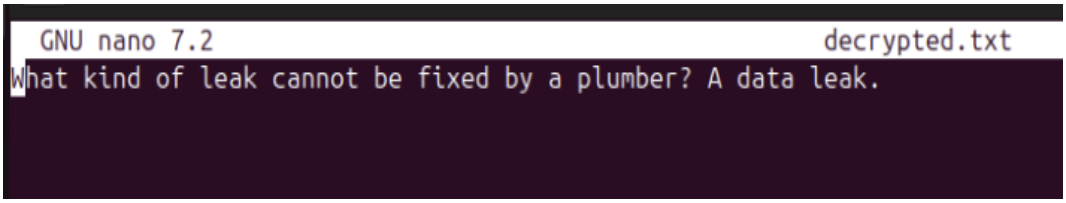
   Now, for the 3-key 3DES, extending the attack becomes even more difficult since we have to deal with 3 independent keys now, which increases the effective key space and the number of intermediate values that must be stored and compared. As a result, the computational effort, memory requirements, and the number of plaintext-ciphertext pairs that are needed for a successful attack increase

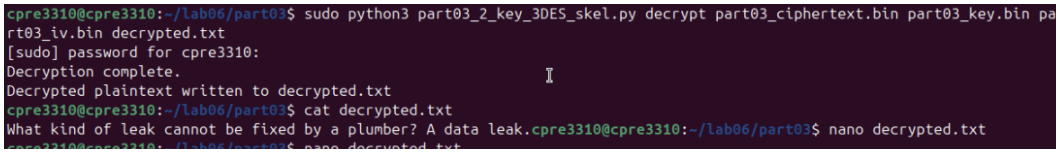
significantly, making practical attacks much harder against 2DES.

8. **Upload your code as `<netid>_part02_mitm_2DES.py`**
   (10 points total)

Part03

9. **Include a screenshot of the plaintext result in your lab report.**
   (5 points)





10. **What is the flaw with the block cipher mode you are using in this implementation of Triple DES?**
    (5 points)

    Over here, the implementation basically uses the cipher block chaining mode. Although, CBC hides plaintext better than the ECB, it still has a flaw, which is that it basically provides confidentiality but not integrity or authentication, which means that an attacker could modify the ciphertext blocks, which would cause predictabel changes in the decrypted plaintext, so without a proper authentication mechanism, CBC is vulnerable to certain attacks such as bit-flipping attacks and padding oracle attacks.

11. **In 2-key 3DES, what is the *effective key length* for encryption? How does it compare to the effective key length of 3-key 3DES? What are those other bits used for?**
    (15 points total, 5 points each question)

    Ans) In 2-key 3DES, basically 2 independent DES keys are used, which gives an effective key length of 112 bits, and that's because each DES key has 56 bits of the actual key material.

    Now, in 3-key 3DES, 3 independent keys are used, resulting in an effective key length of 168 bits (which is basically 56 bits x 3 keys).

    The DES keys are actually 64 bits long, but only 56 of those bits are used for encryption. The remaining 8 bits are parity bits used for error detection to verify

whether the key was transmitted correctly, and they are not part of the encryption key itself.

12. **Historically, has there been a way to use a single key in Triple DES where you set key1=key2=key3? Why would you want to do that? What is the effective key size when key1=key2=key3?**
(15 points total, 5 points each question)

    Ans) Yes, historically Triple DES could be configured with key1 = key2 = key3, which effectively uses a single key. This configuration was basically used for backward compatibility with systems that already used DES, which allows those systems to operate within a Triple DES framework within a Triple DES framework without changing their existing key infrastructure and when all 3 keys are same, then Triple DES simplifies to the equivalent of single DES encryption since the encrypt-decrypt-encrypt sequence cancles out the additional layers, and as a result the effective key size in this configuration is 56 bits, which is same as the standard DES.

13. **Upload your code `<netid>_part03_2_key_3DES.py`**
(5 points)